Sesión 12

Bases de Datos Vectoriales y RAG

Carlos César Sánchez Coronel

BASES DE DATOS

2026

Contenido general

1. Introducción a las Bases de Datos Vectoriales
2. Embeddings
3. Búsqueda de Similitud: k-NN y ANN
4. Métricas de Distancia
5. Tecnologías de Bases de Datos Vectoriales
6. Retrieval Augmented Generation (RAG)
7. Integración con SQL y NoSQL
8. Despliegue On-Premise y Cloud
9. Casos de Uso Reales
10. Ejercicios Resueltos
11. Glosario

Contenido de la sección

1. Introducción a las Bases de Datos Vectoriales
2. Embeddings
3. Búsqueda de Similitud: k-NN y ANN
4. Métricas de Distancia
5. Tecnologías de Bases de Datos Vectoriales
6. Retrieval Augmented Generation (RAG)
7. Integración con SQL y NoSQL
8. Despliegue On-Premise y Cloud
9. Casos de Uso Reales
10. Ejercicios Resueltos
11. Glosario

¿Qué es un vector en el contexto de IA?

- Un **embedding** es una representación densa de baja dimensionalidad de un objeto (texto, imagen, audio, etc.) en un espacio continuo.
- Objetos semánticamente similares tienen vectores cercanos según alguna métrica.
- Generados por modelos como Word2Vec, BERT, Sentence Transformers, CLIP.

Limitaciones de las bases de datos tradicionales

- No están optimizadas para búsquedas por similitud semántica.
- Búsquedas exactas o por palabras clave (LIKE, full-text) no capturan significado.
- No manejan eficientemente vectores de alta dimensionalidad.
- Escaneo completo (O(n)) inviable para millones de vectores.

¿Qué hace una base de datos vectorial?

- Almacena vectores junto con metadatos.
- Indexa los vectores para búsquedas rápidas de vecinos cercanos (ANN).
- Ofrece APIs para calcular distancias y recuperar elementos similares.

Contenido de la sección

1. Introducción a las Bases de Datos Vectoriales
2. Embeddings
3. Búsqueda de Similitud: k-NN y ANN
4. Métricas de Distancia
5. Tecnologías de Bases de Datos Vectoriales
6. Retrieval Augmented Generation (RAG)
7. Integración con SQL y NoSQL
8. Despliegue On-Premise y Cloud
9. Casos de Uso Reales
10. Ejercicios Resueltos
11. Glosario

Definición y generación

- Un **embedding** es una lista ordenada de números que representa un objeto.
- Modelos populares:
  - Texto: Sentence Transformers (all-MiniLM-L6-v2), OpenAI embeddings.
  - Imágenes: CLIP, ResNet.
  - Multimodal: CLIP, ImageBind.

Ejemplo con Sentence Transformers

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')
oraciones = ["El gato duerme en el sofá", "Un perro juega en el parque"]
embeddings = model.encode(oraciones)
print(embeddings.shape)  # (2, 384)
print(embeddings[0][:5]) # primeros 5 valores del primer vector
```

Contenido de la sección

1. Introducción a las Bases de Datos Vectoriales
2. Embeddings
3. Búsqueda de Similitud: k-NN y ANN
4. Métricas de Distancia
5. Tecnologías de Bases de Datos Vectoriales
6. Retrieval Augmented Generation (RAG)
7. Integración con SQL y NoSQL
8. Despliegue On-Premise y Cloud
9. Casos de Uso Reales
10. Ejercicios Resueltos
11. Glosario

k-NN exacto vs ANN

k-NN exacto Calcula distancia con todos los vectores, O(n) por consulta. Inviable para millones.

ANN (Approximate Nearest Neighbors) Algoritmos que sacrifican precisión por velocidad (complejidad sublineal).

Algoritmos ANN

HNSW Grafos multinivel, alta velocidad y precisión, alto consumo de memoria.

IVF Divide espacio en clusters (k-means), explora clusters cercanos.

PQ (Product Quantization) Comprime vectores, reduce memoria, usado con IVF.

Comparativa de algoritmos

Algoritmo	Velocidad	Precisión
HNSW	Alta	Muy alta
IVF	Alta	Alta
PQ	Muy alta	Media

Contenido de la sección

1. Introducción a las Bases de Datos Vectoriales
2. Embeddings
3. Búsqueda de Similitud: k-NN y ANN
4. Métricas de Distancia
5. Tecnologías de Bases de Datos Vectoriales
6. Retrieval Augmented Generation (RAG)
7. Integración con SQL y NoSQL
8. Despliegue On-Premise y Cloud
9. Casos de Uso Reales
10. Ejercicios Resueltos
11. Glosario

Métricas comunes

Similitud de coseno Mide ángulo entre vectores; ignora magnitudes. $\frac{u \cdot v}{\|u\| \|v\|}$

Distancia Euclidiana (L2) Distancia geométrica: $\sqrt{\sum (u_i - v_i)^2}$

Producto punto Si vectores normalizados, equivale al coseno.

Importante

Usar la métrica con la que fue entrenado el modelo (ej. coseno para Sentence Transformers).

Contenido de la sección

1. Introducción a las Bases de Datos Vectoriales
2. Embeddings
3. Búsqueda de Similitud: k-NN y ANN
4. Métricas de Distancia
5. Tecnologías de Bases de Datos Vectoriales
6. Retrieval Augmented Generation (RAG)
7. Integración con SQL y NoSQL
8. Despliegue On-Premise y Cloud
9. Casos de Uso Reales
10. Ejercicios Resueltos
11. Glosario

Bases de datos nativas

Pinecone	Servicio cloud gestionado, índice HNSW.
Milvus	Open source, múltiples índices, despliegue on-premise/cloud.
Weaviate	Open source, módulos de ML, búsqueda híbrida.
Qdrant	Alto rendimiento en Rust, open source y cloud.
Chroma	Ligera, para prototipos y proyectos pequeños.

Extensiones sobre bases de datos existentes

pgvector	Extensión PostgreSQL, índices IVFFlat y HNSW.
Elasticsearch	Tipo dense_vector, búsqueda híbrida.
Redis Stack	Módulo RedisVL para búsqueda vectorial.
MongoDB Atlas Search	Soporte vectorial (beta).

Tabla comparativa

Tecnología	Tipo	Índices
Pinecone	Nativa (cloud)	HNSW
Milvus	Nativa (open source)	HNSW, IVF, etc.
Weaviate	Nativa (open source)	HNSW
pgvector	Extensión PostgreSQL	IVFFlat, HNSW
Elasticsearch	Motor de búsqueda	HNSW (plugin)
RedisVL	Módulo Redis	HNSW

Ejemplo con pgvector

```sql
CREATE EXTENSION vector;

CREATE TABLE productos (
    id SERIAL PRIMARY KEY,
    nombre TEXT,
    descripcion TEXT,
    embedding vector(384)
);

-- Indice HNSW (requiere pgvector >= 0.5.0)
CREATE INDEX ON productos USING hnsw (embedding vector_cosine_ops);

-- Búsqueda: productos similares a una descripción
SELECT nombre, embedding <=> '[0.1,0.2,...]' AS distancia
FROM productos
ORDER BY embedding <=> '[0.1,0.2,...]'
LIMIT 10;
```

Contenido de la sección

1. Introducción a las Bases de Datos Vectoriales
2. Embeddings
3. Búsqueda de Similitud: k-NN y ANN
4. Métricas de Distancia
5. Tecnologías de Bases de Datos Vectoriales
6. Retrieval Augmented Generation (RAG)
7. Integración con SQL y NoSQL
8. Despliegue On-Premise y Cloud
9. Casos de Uso Reales
10. Ejercicios Resueltos
11. Glosario

Problema que resuelve RAG

- Los LLMs tienen conocimiento limitado a su fecha de entrenamiento.
- Pueden alucinar (inventar) cuando no saben algo.
- RAG proporciona contexto relevante extraído de una base de conocimiento, reduciendo alucinaciones.

Flujo de trabajo típico

1. **Indexación**: dividir documentos, generar embeddings, almacenar en BD vectorial.
2. **Consulta**: usuario hace pregunta.
3. **Embedding de consulta** y búsqueda de fragmentos similares.
4. **Generación de prompt** con fragmentos recuperados.
5. **LLM** genera respuesta basada en el contexto.

Orquestadores para RAG

LangChain Framework para cadenas con LLMs, integración con múltiples fuentes y bases vectoriales.

LlamaIndex Especializado en indexación y recuperación de datos.

Haystack Framework modular para búsqueda y RAG.

Ejemplo con LangChain y Chroma

```python
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma
from langchain.llms import OpenAI
from langchain.chains import RetrievalQA

# Cargar y dividir PDF
loader = PyPDFLoader("documento.pdf")
documentos = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
docs = splitter.split_documents(documentos)

# Embeddings y vectorstore
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = Chroma.from_documents(docs, embeddings)

# Cadena QA
llm = OpenAI(temperature=0)
qa = RetrievalQA.from_chain_type(llm=llm, retriever=vectorstore.as_retriever())
respuesta = qa.run(" Cu ál es el tema principal del documento?")
print(respuesta)
```

Contenido de la sección

1. Introducción a las Bases de Datos Vectoriales
2. Embeddings
3. Búsqueda de Similitud: k-NN y ANN
4. Métricas de Distancia
5. Tecnologías de Bases de Datos Vectoriales
6. Retrieval Augmented Generation (RAG)
7. Integración con SQL y NoSQL
8. Despliegue On-Premise y Cloud
9. Casos de Uso Reales
10. Ejercicios Resueltos
11. Glosario

Casos de integración

- Metadatos en PostgreSQL, embeddings en pgvector (búsqueda híbrida).
- Documentos en MongoDB, búsqueda semántica con base vectorial externa.
- Data lake con archivos Parquet + catálogo de embeddings.

Estrategias de consistencia

- **Sincronización síncrona**: escribir en ambas bases dentro de transacción.
- **CDC (Change Data Capture)**: capturar cambios y propagar vía Kafka/Debezium.
- **Batch periódico**: reindexar cada cierto tiempo.

Búsqueda híbrida con pgvector

```sql
SELECT nombre, precio, embedding <=> query_embedding AS distancia
FROM productos
WHERE categoria = 'electronica' AND precio < 500
ORDER BY embedding <=> query_embedding
LIMIT 10;
```

Contenido de la sección

1. Introducción a las Bases de Datos Vectoriales
2. Embeddings
3. Búsqueda de Similitud: k-NN y ANN
4. Métricas de Distancia
5. Tecnologías de Bases de Datos Vectoriales
6. Retrieval Augmented Generation (RAG)
7. Integración con SQL y NoSQL
8. Despliegue On-Premise y Cloud
9. Casos de Uso Reales
10. Ejercicios Resueltos
11. Glosario

On-Premise vs Cloud

On-Premise Control total, seguridad, sin dependencia de proveedor. Mantenimiento y escalado manual.

Cloud Escalabilidad elástica, menor overhead operativo, pago por uso. Dependencia del proveedor, costos variables.

Servicios cloud gestionados

- **AWS**: OpenSearch Serverless (vectorial), Aurora PostgreSQL con pgvector, MemoryDB (Redis).
- **GCP**: Vertex AI Vector Search, AlloyDB con pgvector.
- **Azure**: Cognitive Search con vectorial, Cosmos DB (preview).

Dimensionamiento y costos

- Factores: número de vectores, dimensionalidad, tipo de índice, QPS.
- Ejemplo Pinecone: $0{,}50/horaporcadamillndevectores(aprox).Para10Mvectores, costomensual3{,}600$.
- pgvector en RDS puede ser más económico si ya se tiene instancia.

Contenido de la sección

1. Introducción a las Bases de Datos Vectoriales
2. Embeddings
3. Búsqueda de Similitud: k-NN y ANN
4. Métricas de Distancia
5. Tecnologías de Bases de Datos Vectoriales
6. Retrieval Augmented Generation (RAG)
7. Integración con SQL y NoSQL
8. Despliegue On-Premise y Cloud
9. Casos de Uso Reales
10. Ejercicios Resueltos
11. Glosario

Búsqueda semántica en e-commerce

Usuarios buscan con lenguaje natural (ej. “vestido rojo elegante”). Embeddings de descripciones vs. embedding de consulta.

Sistemas de preguntas y respuestas (FAQ)

Documentación interna indexada en base vectorial; chatbot RAG responde preguntas de empleados.

Recomendación de contenidos

Embeddings de películas basados en trama/género; recomendar las más similares a las vistas por el usuario.

Detección de plagio / documentos duplicados

Comparar embeddings de documentos para encontrar similitudes.

Búsqueda multimodal

Usar CLIP para buscar imágenes con texto o viceversa.

Contenido de la sección

1. Introducción a las Bases de Datos Vectoriales
2. Embeddings
3. Búsqueda de Similitud: k-NN y ANN
4. Métricas de Distancia
5. Tecnologías de Bases de Datos Vectoriales
6. Retrieval Augmented Generation (RAG)
7. Integración con SQL y NoSQL
8. Despliegue On-Premise y Cloud
9. Casos de Uso Reales
10. Ejercicios Resueltos
11. Glosario

Ejercicio 1: Embeddings y pgvector

Enunciado: Generar embeddings de oraciones y buscar similitud con pgvector.

```
import psycopg2
from sentence_transformers import SentenceTransformer

conn = psycopg2.connect(dbname="test", user="postgres")
cur = conn.cursor()
cur.execute("CREATE EXTENSION IF NOT EXISTS vector;")
cur.execute("DROP TABLE IF EXISTS oraciones;")
cur.execute("CREATE TABLE oraciones (id SERIAL PRIMARY KEY, texto TEXT, embedding vector(384));")

model = SentenceTransformer('all-MiniLM-L6-v2')
oraciones = ["El gato duerme en el sofá", "Los perros juegan en el parque", "Me gusta la comida
    italiana", "El cielo está nublado hoy"]
embeddings = model.encode(oraciones).tolist()

for texto, emb in zip(oraciones, embeddings):
    cur.execute("INSERT INTO oraciones (texto, embedding) VALUES (%s, %s);", (texto, emb))
conn.commit()

consulta = "animales domésticos"
emb_consulta = model.encode([consulta]).tolist()[0]
cur.execute("""SELECT texto, embedding <=> %s::vector AS distancia FROM oraciones ORDER BY
    embedding <=> %s::vector LIMIT 2;""", (emb_consulta, emb_consulta))
for texto, dist in cur.fetchall():
    print(f"{texto} (distancia: {dist:.4f})")
cur.close(); conn.close()
```

Ejercicio 2: Pipeline RAG con LangChain y Chroma

Enunciado: Cargar PDF, dividir, crear embeddings, construir QA con Chroma.

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma
from langchain.llms import OpenAI
from langchain.chains import RetrievalQA

loader = PyPDFLoader("documento.pdf")
documentos = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
docs = splitter.split_documents(documentos)

embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = Chroma.from_documents(docs, embeddings)

llm = OpenAI(temperature=0)
qa = RetrievalQA.from_chain_type(llm=llm, retriever=vectorstore.as_retriever())
respuesta = qa.run(" Cual  es el tema principal del documento?")
print(respuesta)
```

Ejercicio 3: Búsqueda híbrida con pgvector

Enunciado: Filtrar por precio y categoría, ordenar por similitud.

```
import psycopg2
from sentence_transformers import SentenceTransformer

conn = psycopg2.connect(dbname="test", user="postgres")
cur = conn.cursor()

model = SentenceTransformer('all-MiniLM-L6-v2')
consulta = "teléfono inteligente con buena cámara"
emb_consulta = model.encode([consulta]).tolist()[0]

cur.execute("""
    SELECT nombre, precio, embedding <=> %s::vector AS dist
    FROM productos
    WHERE precio > 50 AND categoria = 'electronica'
    ORDER BY embedding <=> %s::vector
    LIMIT 5;
""", (emb_consulta, emb_consulta))

for row in cur.fetchall():
    print(row)
cur.close(); conn.close()
```

Ejercicio 4: Comparación de índices en Milvus

Enunciado: Crear colección, insertar vectores, comparar HNSW vs IVFFlat.

```
from pymilvus import connections, Collection, FieldSchema, CollectionSchema, DataType, utility
import numpy as np
import time

connections.connect("default", host="localhost", port="19530")

fields = [
    FieldSchema(name="id", dtype=DataType.INT64, is_primary=True),
    FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=128)
]
schema = CollectionSchema(fields)
collection = Collection("test_vector", schema)

data = [
    [i for i in range(10000)],
    [np.random.rand(128).tolist() for _ in range(10000)]
]
collection.insert(data)
collection.flush()

# HNSW
hnsw_params = {"index_type": "HNSW", "metric_type": "COSINE", "params": {"M": 16, "efConstruction
    ": 200}}
collection.create_index("embedding", hnsw_params)
collection.load()
start = time.time()
```

Contenido de la sección

1. Introducción a las Bases de Datos Vectoriales
2. Embeddings
3. Búsqueda de Similitud: k-NN y ANN
4. Métricas de Distancia
5. Tecnologías de Bases de Datos Vectoriales
6. Retrieval Augmented Generation (RAG)
7. Integración con SQL y NoSQL
8. Despliegue On-Premise y Cloud
9. Casos de Uso Reales
10. Ejercicios Resueltos
11. Glosario

Términos Clave

Embedding Representación vectorial de un objeto.

Vector Lista ordenada de números.

Dimensionalidad Número de componentes del vector.

Similitud de coseno Medida del ángulo entre vectores.

Distancia Euclidiana Distancia geométrica.

ANN Approximate Nearest Neighbors.

HNSW Hierarchical Navigable Small World.

IVF Inverted File Index.

PQ Product Quantization.

RAG Retrieval Augmented Generation.

LangChain Framework para aplicaciones con LLMs.

pgvector Extensión vectorial para PostgreSQL.

Pinecone Base de datos vectorial gestionada.

Milvus Plataforma de búsqueda vectorial open source.

¿Preguntas?

¡Gracias por su atención!